

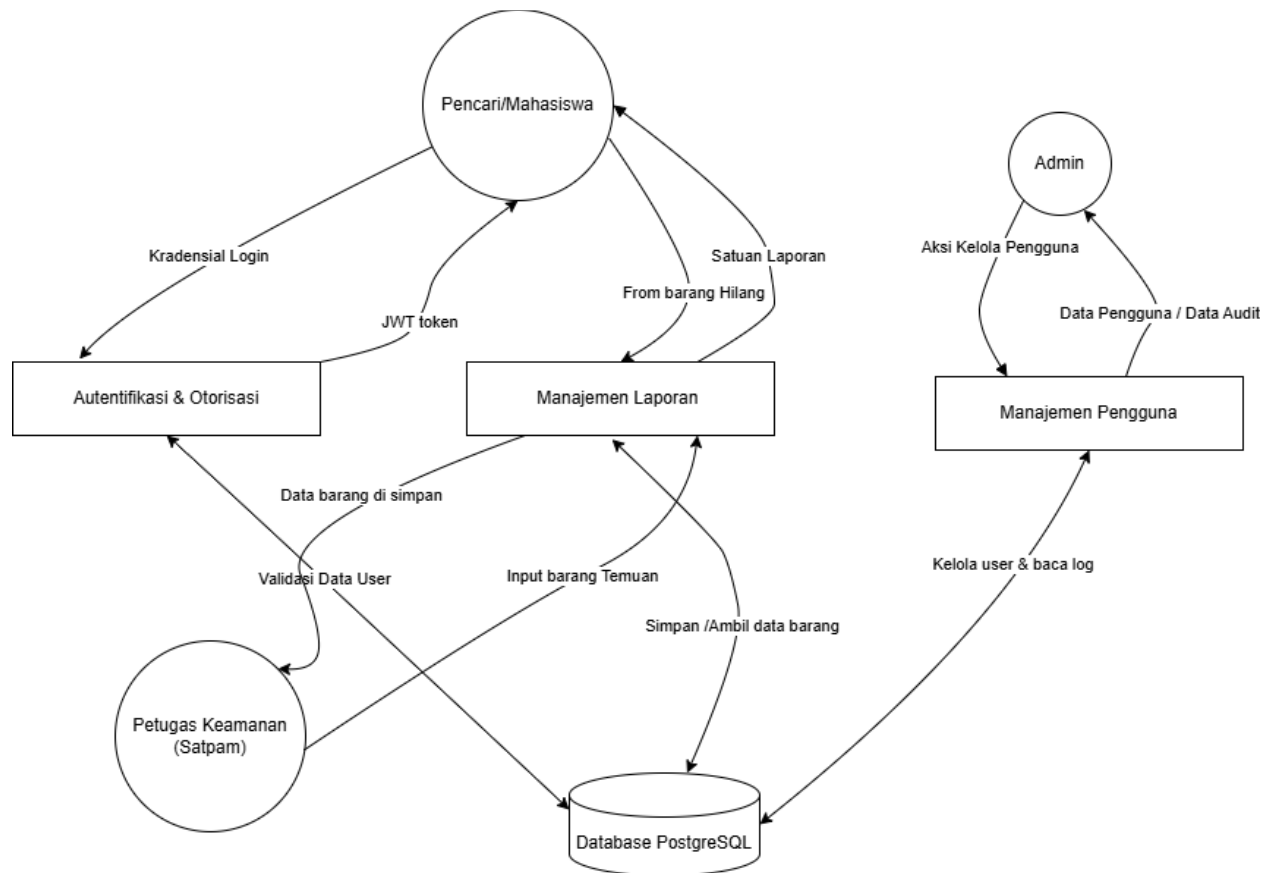
Threat Modeling

1. Deskripsi Singkat

IPB Lost & Found Portal atau Balikin adalah sebuah platform digital terpusat yang dirancang khusus untuk mengatasi permasalahan pelaporan dan pencarian barang hilang di lingkungan kampus Institut Pertanian Bogor (IPB). Sistem ini menggantikan metode pelaporan informal (seperti grup WhatsApp atau platform X) menjadi sistem yang lebih terstruktur, terdokumentasi, dan mudah diakses oleh seluruh civitas akademik.

2. Data Flow Diagram

Berikut adalah Data Flow Diagram (DFD) Level 1 yang menggambarkan aliran data antara entitas (Pencari, Petugas, Admin) dan proses utama di dalam sistem.



3. Identifikasi Ancaman (Metode STRIDE)

Metodologi STRIDE adalah kerangka kerja sistematis dari Microsoft yang digunakan untuk mengidentifikasi ancaman keamanan pada sebuah sistem melalui enam kategori: Spoofing (penyamaran identitas), Tampering (perubahan data ilegal), Repudiation (penyangkalan tindakan), Information Disclosure (kebocoran data), Denial of Service (gangguan ketersediaan sistem), dan

Elevation of Privilege (peningkatan hak akses secara tidak sah). Dengan menerapkan metode ini pada fase desain, pengembang dapat memetakan celah keamanan secara proaktif dan merancang mitigasi yang tepat untuk memastikan integritas, kerahasiaan, dan ketersediaan sistem yang dikembangkan.

Tabel Identifikasi Ancaman (STRIDE)

Komponen Sistem	Ancaman (Threat)	Kategori (STRIDE)	Dampak	Mitigasi
API Server	Penyerang menebak kredensial pengguna, melakukan <i>brute-force</i> , atau menggunakan <i>JWT token</i> curian untuk membajak sesi akun pengguna lain.	Spoofing	Pengambilalihan akun, penipuan klaim barang orang lain.	Terapkan kebijakan <i>password</i> kuat, batasi percobaan login (<i>rate limiting</i>), gunakan masa berlaku (<i>expiration</i>) singkat pada <i>JWT token</i> , dan terapkan <i>refresh token</i> .
Database / API	Serangan injeksi (<i>SQL Injection</i>) pada saat mencari barang hilang atau saat menyimpan input form laporan.	Tampering	Perubahan atau penghapusan data laporan dan pengguna di <i>database</i> secara ilegal.	Gunakan ORM bawaan (<i>SQLAlchemy</i>) secara ketat dan selalu validasi input menggunakan skema <i>Pydantic</i> . Jangan pernah menyusun <i>query string</i> secara manual.
Web Frontend	Serangan <i>Cross-Site Scripting (XSS)</i> yang disisipkan melalui deskripsi "Barang Hilang" oleh pengguna.	Tampering / Info Disclosure	Pencurian token sesi (jika disimpan di <i>localStorage</i>), dan eksekusi <i>script</i> berbahaya pada <i>browser</i> pengguna lain.	Lakukan sanitasi input dan <i>output</i> (React.js secara <i>default</i> sudah aman terhadap XSS, namun hindari penggunaan <i>dangerouslySetInnerHTML</i>). Simpan <i>JWT</i> di <i>HttpOnly Cookie</i> alih-alih <i>localStorage</i> .
API Server	Petugas secara tidak sengaja menyerahkan barang kepada orang yang salah, lalu menyangkal	Repudiation	Hilangnya akuntabilitas dan kepercayaan terhadap sistem serta petugas.	Buat sistem <i>audit log</i> terpusat (menyimpan <i>user_id</i> , waktu, dan detail aksi) untuk setiap tindakan kritis (klaim, hapus, verifikasi) yang

	telah memverifikasi laporan tersebut.			bersifat <i>append-only</i> (tidak bisa diubah).
Jaringan	Komunikasi <i>client</i> (aplikasi web) dan <i>server</i> (API) disadap pada jaringan Wi-Fi publik (misal: Wi-Fi kampus).	Information Disclosure	Kredensial, JWT token, dan identitas/nomor kontak pelapor dapat dicuri oleh <i>hacker</i> (serangan Man-in-the-Middle).	Wajibkan penggunaan HTTPS (enkripsi SSL/TLS) untuk seluruh komunikasi antara Frontend, API, dan Database di tahap produksi.
API Server	Serangan bot yang membanjiri API (misal: endpoint pencarian atau daftar barang) dengan <i>request</i> massal.	Denial of Service	Server <i>down</i> (lumpuh), membuat <i>database</i> lambat sehingga tidak bisa digunakan oleh civitas IPB.	Implementasikan <i>API Rate Limiting</i> di FastAPI dan gunakan Web Application Firewall (WAF) seperti Cloudflare pada tahap <i>deployment</i> .
API Server	Mahasiswa (<i>Pencari</i>) memanipulasi <i>request</i> API untuk mengakses fungsi verifikasi klaim atau panel kelola <i>user</i> milik Admin.	Elevation of Privilege	Akses fungsi kritikal secara tidak sah, memungkinkan mahasiswa mengklaim barangnya sendiri tanpa melalui Satpam.	Validasi ketat hak akses atau peran (<i>Role-Based Access Control/RBAC</i>) di setiap <i>endpoint</i> level Backend (FastAPI). Jangan hanya mengandalkan penyembunyian tombol UI di <i>Frontend</i>).

4. Analisis Kerentanan

Dari berbagai ancaman di atas, celah yang paling berbahaya dan probabilitas terjadinya paling tinggi pada arsitektur sistem ini adalah:

1. Broken Access Control & Insecure Direct Object Reference (IDOR)

Karena sistem ini membedakan peran menjadi Pencari, Petugas, dan Admin, kesalahan umum dalam pengembangan API (seperti FastAPI) adalah pengembang hanya memverifikasi bahwa token valid, namun gagal memverifikasi apakah pemegang token tersebut memiliki izin khusus untuk fungsi atau objek tertentu.

Skenario Eksploitasi:

- Pengguna A (Pencari) melakukan modifikasi request PUT /api/items/123/verify (endpoint petugas) menggunakan JWT token miliknya yang sah.

- Atau, pengguna A mengubah ID di URL (misal: DELETE /api/items/555) yang sebenarnya merupakan barang milik Pengguna B. Jika Backend tidak memeriksa kepemilikan (WHERE user_id = A), barang milik B akan terhapus.

Dampak nya seseorang dapat dengan mudah mengklaim barang yang bukan miliknya, memintas validasi keamanan satpam (Petugas), atau bahkan mengakses log sistem milik Admin yang berujung pada kekacauan data laporan hilang temu.

2. Penyimpanan JWT Token yang Kurang Aman (Insecure Client-Side Storage)

Secara umum, aplikasi React (Single Page Application) cenderung menyimpan JWT token pada localStorage.

Skenario Eksploitasi: Jika terdapat satu celah XSS (misalnya di fitur komentar atau deksripsi barang yang tidak tersanitasi), token JWT yang ada di localStorage dapat langsung dicuri menggunakan *JavaScript payload*.

Dampak: *Session Hijacking* (Pembajakan sesi). Peretas dapat bertindak atas nama pengguna asli tanpa membutuhkan *password*.

5. Kesimpulan & Rekomendasi

Sistem **Balikin** menggunakan *tech stack* modern yang sudah memberikan keamanan *built-in* (seperti anti-SQLi dengan SQLAlchemy dan anti-XSS dengan React). Meski demikian, celah pada *business logic* sering kali terabaikan.

Tindakan Preventif:

1. Penguatan Otorisasi Backend (RBAC & IDOR check): Setiap *endpoint* yang memanipulasi data di FastAPI harus memiliki dua lapis keamanan:
 - Lapis 1: Memeriksa token (*Authentication*).
 - Lapis 2: Memeriksa peran dan hak akses pemegang token terhadap data tersebut (*Authorization*).
2. Keamanan Transpor dan Sesi: Pastikan tidak ada HTTP polos (wajib HTTPS di produksi) dan pertimbangkan menggunakan *HttpOnly Cookie* untuk menyimpan *access token*, bukan localStorage.
3. Pencatatan yang Akuntabel (Audit Trail): Tambahkan tabel *audit_logs* di database. Setiap pergantian status barang (dilaporkan, ditemukan, dikembalikan) harus mencatat waktu dan *user_id* pelaksana, karena sering terjadi penyangkalan (*repudiation*) jika barang fisik ternyata salah diberikan.

4. Validasi Skema Ketat: Gunakan validator Pydantic secara maksimal (contoh: batasi panjang karakter, blokir karakter *script*, pastikan rentang angka valid) agar data *garbage/malicious* tertolak sebelum masuk tahap logika *database*.